

1. Install mongodb-mongosh

```
Tilix: asaad@asus: ~  
1: asaad@asus: ~  
asaad@asus:~$ mongod --version  
db version v8.2.3  
Build Info: {  
  "version": "8.2.3",  
  "gitVersion": "36f41c9c30a2f13f834d033ba03c3463c891fb01",  
  "opensslVersion": "OpenSSL 3.0.2 15 Mar 2022",  
  "modules": [],  
  "allocator": "tcmalloc-google",  
  "environment": {  
    "distmod": "ubuntu2204",  
    "distarch": "x86_64",  
    "target_arch": "x86_64"  
  }  
}  
asaad@asus:~$ mongosh -version  
2.6.0  
asaad@asus:~$
```

2. Start a server using mongod

```
Tilix: asaad@asus: ~  
1: asaad@asus: ~  
asaad@asus:~$ sudo systemctl start mongod  
asaad@asus:~$
```

3. Use mongosh to connect to the server

```
1/1 + ... action=true&serverSelectionTimeoutMS=2000
1: asaad@asus: ~
asaad@asus:~$ mongosh
Current Mongosh Log ID: 696cd0a563be094a3a8ce5af
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSe
lectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB:      8.2.3
Using Mongosh:       2.6.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-01-18T14:21:59.018+02:00: Using the XFS filesystem is strongly recommende
d with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnote
s-filesystem
2026-01-18T14:21:59.415+02:00: Access control is not enabled for the database.
Read and write access to data and configuration is unrestricted
2026-01-18T14:21:59.415+02:00: For customers running the current memory alloca
tor, we suggest changing the contents of the following sysfsFile
2026-01-18T14:21:59.415+02:00: For customers running the current memory alloca
tor, we suggest changing the contents of the following sysfsFile
2026-01-18T14:21:59.415+02:00: We suggest setting the contents of sysfsFile to
0.
2026-01-18T14:21:59.415+02:00: We suggest setting swappiness to 0 or 1, as swa
pping can cause performance problems.
-----
test> █
```

4. Create a db named iti

```
test> use ITI
switched to db ITI
ITI> █
```

5. Create a variable with the data

```
ITI> let instructorsArray = [{
...   _id: 6, firstName: "noha", lastName: "hesham",
...   age: 21, salary: 3500,
...   address: { city: "cairo", street: 10, building: 8 },
...   courses: ["js", "mvc", "signalR", "expressjs"]
... },
... {
...   _id: 7, firstName: "mona", lastName: "ahmed",
...   age: 21, salary: 3600,
...   address: { city: "cairo", street: 20, building: 8 },
...   courses: ["es6", "mvc", "signalR", "expressjs"]
... },
... {
...   _id: 8, firstName: "mazen", lastName: "mohammed",
...   age: 21, salary: 7040,
...   address: { city: "Ismailia", street: 10, building: 8 },
...   courses: ["asp.net", "mvc", "EF"]
... },
... {
...   _id: 9, firstName: "ebtesam", lastName: "hesham",
...   age: 21, salary: 7500,
...   address: { city: "mansoura", street: 14, building: 3 },
...   courses: ["js", "html5", "signalR", "expressjs", "bootstrap"]
... }
... }
```

6. A. Inserting in the db using insertMany

```
ITI> db.instructors.insertMany(instructorsArray);
{
  acknowledged: true,
  insertedIds: { '0': 6, '1': 7, '2': 8, '3': 9, '4': 10, '5': 2, '6': 3, '7': 4 }
}
```

B. Inserting my own data using insertOne

```
ITI> db.instructors.insertOne({_id : 12,"firstName": "Asaad","lastName": "Mansour"});
{ acknowledged: true, insertedId: 12 }
ITI> 
```

C. Insert instructor without firstname and lastname

```
ITI> db.instructors.insertOne({_id : 202,"age": 21});
{ acknowledged: true, insertedId: 202 }
ITI> 
```

It won't complain as it's schemaless db;

7. A. .find({}) : explicit empty object

```
ITI> db.instructors.find({});
[
  {
    _id: 6,
    firstName: 'noha',
    lastName: 'hesham',
    age: 21,
    salary: 3500,
    address: { city: 'cairo', street: 10, building: 8 },
    courses: [ 'js', 'mvc', 'signalR', 'expressjs' ]
  },
  {
    _id: 7,
    firstName: 'mona',
    lastName: 'ahmed',
    age: 21,
    salary: 3600,
    address: { city: 'cairo', street: 20, building: 8 },
    courses: [ 'es6', 'mvc', 'signalR', 'expressjs' ]
  },
  {
    _id: 8,
    firstName: 'mazen',
    lastName: 'mohammed',
    age: 21,
    salary: 7040,
    address: { city: 'Ismailia', street: 10, building: 8 },
  }
]
```

B.find() : implicit empty object

```
ITI> db.instructors.find();
[
  {
    _id: 6,
    firstName: 'noha',
    lastName: 'hesham',
    age: 21,
    salary: 3500,
    address: { city: 'cairo', street: 10, building: 8 },
    courses: [ 'js', 'mvc', 'signalR', 'expressjs' ]
  },
  {
    _id: 7,
    firstName: 'mona',
    lastName: 'ahmed',
    age: 21,
    salary: 3600,
    address: { city: 'cairo', street: 20, building: 8 },
    courses: [ 'es6', 'mvc', 'signalR', 'expressjs' ]
  },
  {
    _id: 8,
    firstName: 'mazen',
    lastName: 'mohammed',
    age: 21,
    salary: 7040,
    address: { city: 'Ismailia', street: 10, building: 8 },
  }
]
```

C. findOne: it returns the first entry

```
ITI> db.instructors.findOne()
{
  _id: 6,
  firstName: 'noha',
  lastName: 'hesham',
  age: 21,
  salary: 3500,
  address: { city: 'cairo', street: 10, building: 8 },
  courses: [ 'js', 'mvc', 'signalR', 'expressjs' ]
}
```

D. [constructor.name](#)

```
ITI> db.users.find().constructor.name
Cursor
```

E.find forEach

```
ITI> db.instructors.find({}).forEach((document)=>print(document));
{
  _id: 6,
  firstName: 'noha',
  lastName: 'hesham',
  age: 21,
  salary: 3500,
  address: { city: 'cairo', street: 10, building: 8 },
  courses: [ 'js', 'mvc', 'signalR', 'expressjs' ]
}
{
  _id: 7,
  firstName: 'mona',
  lastName: 'ahmed',
  age: 21,
  salary: 3600,
  address: { city: 'cairo', street: 20, building: 8 },
  courses: [ 'es6', 'mvc', 'signalR', 'expressjs' ]
}
```

F. Display firstname and lastname only

```
ITI> db.instructors.find({}).forEach((document)=>{print(document.firstName,document.lastName)})
noha hesham
mona ahmed
mazen mohammed
ebtesam hesham
badr ahmed
mona mohammed
mazen ali
ebtesam ahmed
Asaad Mansour
```

G.AVG,MAX,MIN

```
ITI> let sum = 0;
... let count = 0;
... let max = 0;
... let min = Infinity;
...
... db.instructors.find({}).forEach(doc => {
...   if (typeof doc.salary === "number") {
...     sum += doc.salary;
...     count++;
...     if (doc.salary > max) max = doc.salary;
...     if (doc.salary < min) min = doc.salary;
...   }
... });
...
... let avg = sum / count;
...

ITI> print(avg)
5500

ITI> print(max,min)
7500 3500
```